FACULTY OF ENGINEERING
AND TECHNOLOGY
BACHELOR OF TECHNOLOGY

JAVA PROGRAMMING LABORATORY
(03010503PC04)

III SEMESTER

CSE Department

Laboratory Manual
Session 2026-27

CERTIFICATE

This is to certify that

Mr / Ms _____with enrollment no. _____has successfully completed his/her laboratory experiments in the Java Programming Laboratory (03010503PC04) from the department of **COMPUTER SCIENCE & ENGINEERING CSE** during the academic year 2026-2027.

Date of Submission:..... Staff In charge:.....

Head Of Department:.....

TABLE OF CONTENT

12.	Write a Java program to demonstrate inheritance: a) Single Inheritance. b) MultiLevel Inheritance. c) Hierarchial Inheritance. d) Hybrid Inheritance.				
13.	Write a Java program to demonstrate use of the this and super keywords				
14.	Write a Java program to demonstrate polymorphism (method overloading and overriding).				
15.	Write a Java program to implement abstraction using abstract classes and interfaces(Multiple Inheritance).				
16.	Write a Java program to demonstrate exception handling using try-catch-finally blocks.				
17.	Write a Java program to demonstrate exception handling using try-catch-finally blocks.				
18.	Write a Java program to implement List interface using ArrayList and LinkedList.				
19.	Write a Java program to implement Set interface using HashSet and TreeSet				
20.	Write a Java program to implement Queue interface using Priority Queue and Deque.				
21.	Write a Java program to implement map interface using HashMap and LinkedHashMap				
22.	Write a Java program to demonstrate multithreading using Thread class and Runnable interface				

PRACTICAL NO: 01

AIM:

Write a Java program to display "Hello World" and demonstrate the Java program structure and demonstrate use of variables, data types, and type casting.

THEORY:

Java is a high-level, object-oriented programming language in which programs are organized using classes and methods. The execution of a Java application begins from the main() method.

A Java program generally consists of the following elements:

Class – Provides the basic structure in which Java code is written.

Main Method – Acts as the starting point of program execution.

Variables – Named storage locations used to hold values during program execution.

Data Types – Specify the kind of data that a variable can store. Common primitive types include int, double, float, char, boolean, and long.

Type Casting – The conversion of a value from one data type to another. It can be:

Widening Casting: Conversion from a smaller data type to a larger compatible type, performed automatically.

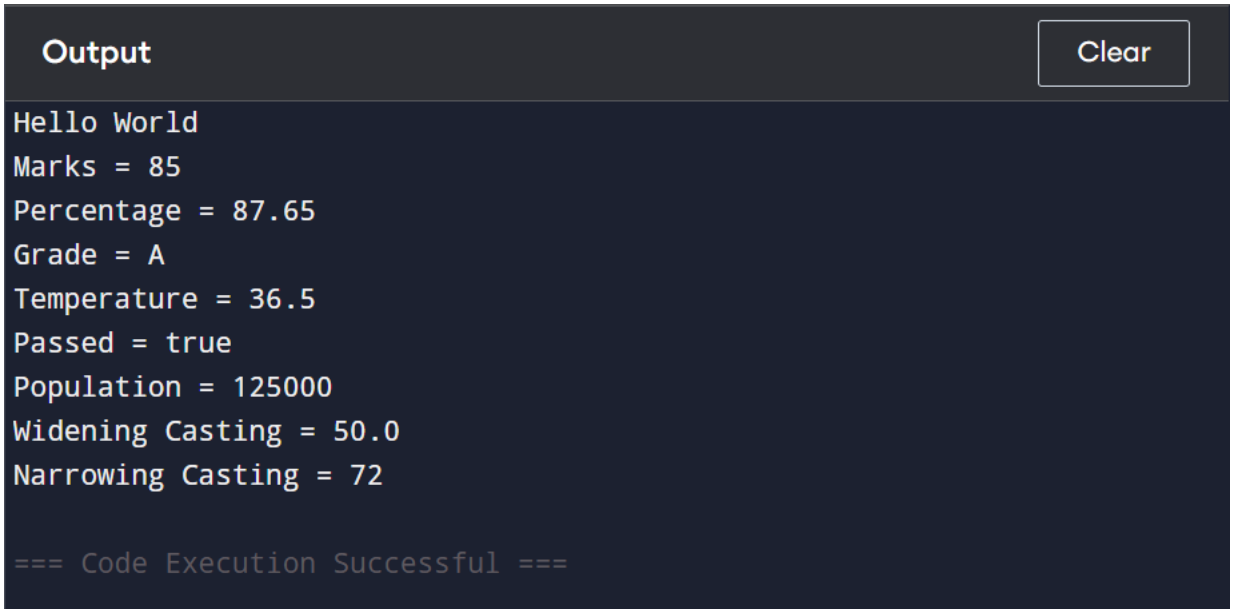
Narrowing Casting: Conversion from a larger data type to a smaller type, performed explicitly using a cast operator.
In this program, a simple message is displayed and different variables are used to demonstrate primitive data types. Both widening and narrowing conversions are also performed.

CODE:

```
class JavaBasics {
  public static void main(String[] args) {

    // Displaying a message
    System.out.println("Hello World");

    // Demonstration of variables and data types
    int marks = 85;
    double percentage = 87.65;
```

```
char grade = 'A';
        float temperature = 36.5f;
        boolean passed = true;
        long population = 125000L;

        System.out.println("Marks = " + marks);
        System.out.println("Percentage = " + percentage);
        System.out.println("Grade = " + grade);
        System.out.println("Temperature = " + temperature);
        System.out.println("Passed = " + passed);
        System.out.println("Population = " + population);

        // Widening type casting
        int number = 50;
        double convertedNumber = number;

        System.out.println("Widening Casting = " + convertedNumber);

        // Narrowing type casting
        double value = 72.89;
        int integerValue = (int) value;

        System.out.println("Narrowing Casting = " + integerValue);
    }
}
```

OUTPUT:



PRACTICAL NO: 02

AIM:

Write a Java program to perform arithmetic, relational, and logical operations, bitwise and other operators.

THEORY:

Operators in Java are symbols that are used to perform different operations on values and variables. They are an important part of expressions and are used for calculations, comparisons, decision-making, and manipulation of individual bits.

The major categories of operators used in this program are:

Arithmetic Operators – Used for mathematical calculations such as addition, subtraction, multiplication, division, and remainder. Examples are +, -, *, /, and %.

Relational Operators – Used to compare two values. The result of a relational operation is either true or false. Examples include ==, !=, >, <, >=, and <=.

Logical Operators – Used to combine multiple conditions. The operators &&, ||, and ! represent logical AND, OR, and NOT respectively.

Bitwise Operators – Work directly on the binary representation of integer values. Operators such as &, |, ^, ~, <<, and >> are used for bit-level operations.

Assignment Operators – Used to assign or update values in variables. Examples include =, +=, -=, *=, and /=.

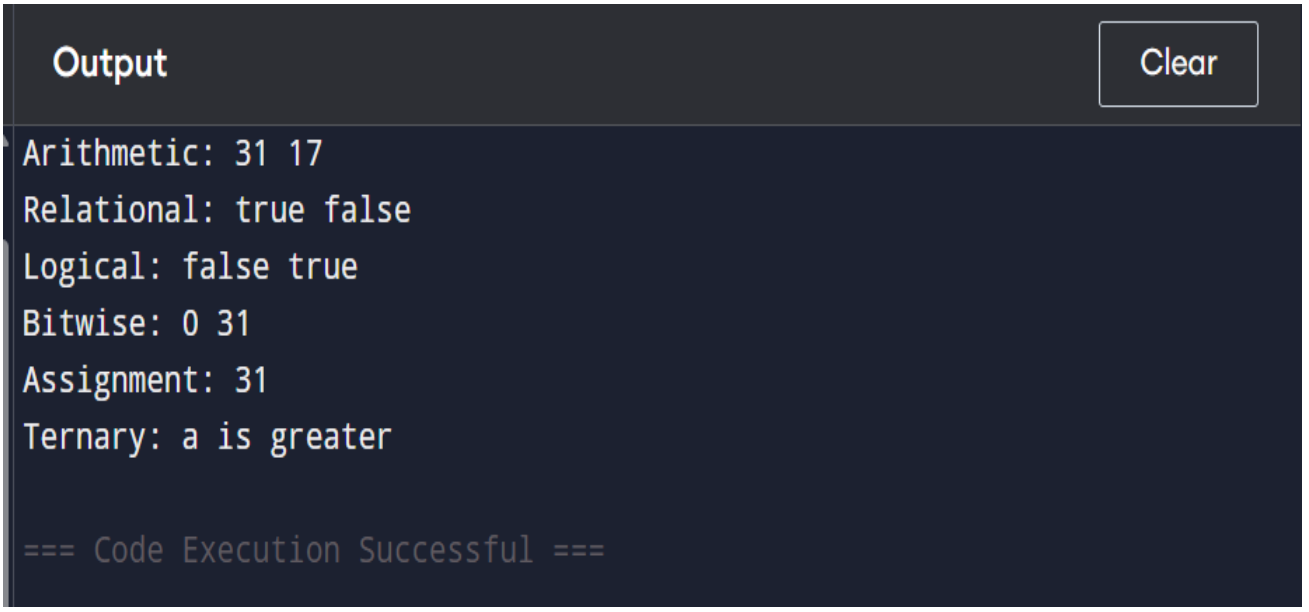
Conditional Operator – The ternary operator ?: provides a short way to select one of two values based on a condition.
The program below demonstrates these operators using different values from the example given in the laboratory manual.

CODE:

```
class OperatorDemo {
  public static void main(String[] args) {

    int a = 24, b = 7;
    boolean x = true, y = false;
```

```
        System.out.println("Arithmetic: " + (a + b) + " " + (a - b));
        System.out.println("Relational: " + (a > b) + " " + (a == b));
        System.out.println("Logical: " + (x && y) + " " + (x || y));
        System.out.println("Bitwise: " + (a & b) + " " + (a | b));

        a += b;
        System.out.println("Assignment: " + a);

        String result = (a > b) ? "a is greater" : "b is greater";
        System.out.println("Ternary: " + result);
    }
}
```

OUTPUT:



PRACTICAL NO: 03

AIM:

Write a Java program using below conditional statements. a) Even or Odd using if.. else statement. b) Roots of Quadratic Equation using else if ladder. c) Largest of three numbers using nested if else. d) Find out the week day using switch statement.

THEORY:

The if...else statement in Java is a conditional control statement used to execute different blocks of code depending on whether a given condition is true or false. It is useful when a program needs to make a decision between two possible outcomes.

In this program, the user enters an integer value, and the modulus operator % is used to determine whether the number is divisible by 2. The modulus operator returns the remainder after division. If the remainder obtained by dividing the number by 2 is 0, the number is considered **even**.
If the remainder is not 0, the number is considered **odd**.

The if block executes when the condition n % 2 == 0 is true, while the else block executes when the condition is false. Thus, the program demonstrates how conditional statements can be used to make simple decisions based on user input.

CODE (A):

```
import java.util.Scanner;

class EvenOdd {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      System.out.print("Enter a number: ");
      int n = sc.nextInt();

      if (n % 2 == 0)
         System.out.println(n + " is Even");
      else
         System.out.println(n + " is Odd");
   }
}
```

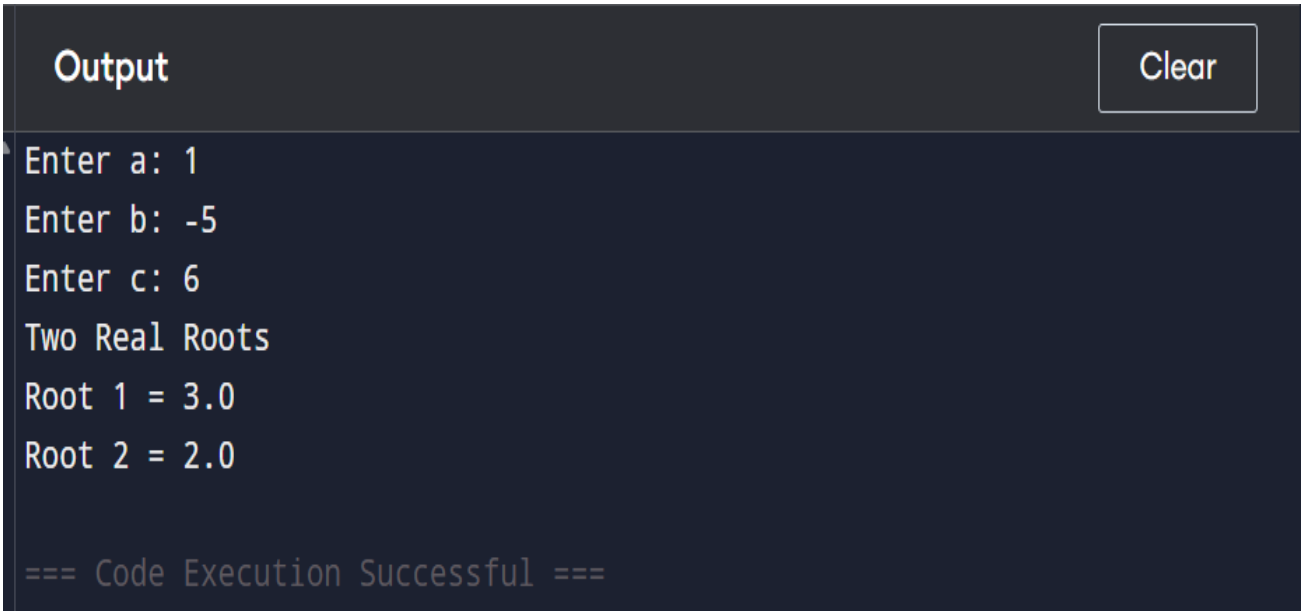
OUTPUT:

CODE (B):

```
import java.util.Scanner;

class Quadratic {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a: ");
        double a = sc.nextDouble();
        System.out.print("Enter b: ");
        double b = sc.nextDouble();
        System.out.print("Enter c: ");
        double c = sc.nextDouble();
        double d = b * b - 4 * a * c;
if (d > 0) {
            double r1 = (-b + Math.sqrt(d)) / (2 * a);
            double r2 = (-b - Math.sqrt(d)) / (2 * a);
```

```
System.out.println("Two Real Roots");
            System.out.println("Root 1 = " + r1);
            System.out.println("Root 2 = " + r2);
        }
        else if (d == 0) {
            double r = -b / (2 * a);
            System.out.println("Equal Roots");
            System.out.println("Root = " + r);
        }
        else {
            System.out.println("Imaginary Roots");
        }
    }
}
```

OUTPUT:



CODE (C):

```
import java.util.Scanner;

class Largest {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter first number: ");
      int a = sc.nextInt();

      System.out.print("Enter second number: ");
      int b = sc.nextInt();

      System.out.print("Enter third number: ");
      int c = sc.nextInt();
      if (a > b) {
         if (a > c)
            System.out.println("Largest = " + a);
         else
            System.out.println("Largest = " + c);
      }
      else {
         if (b > c)
            System.out.println("Largest = " + b);
         else
            System.out.println("Largest = " + c);
      }
   }
}
```

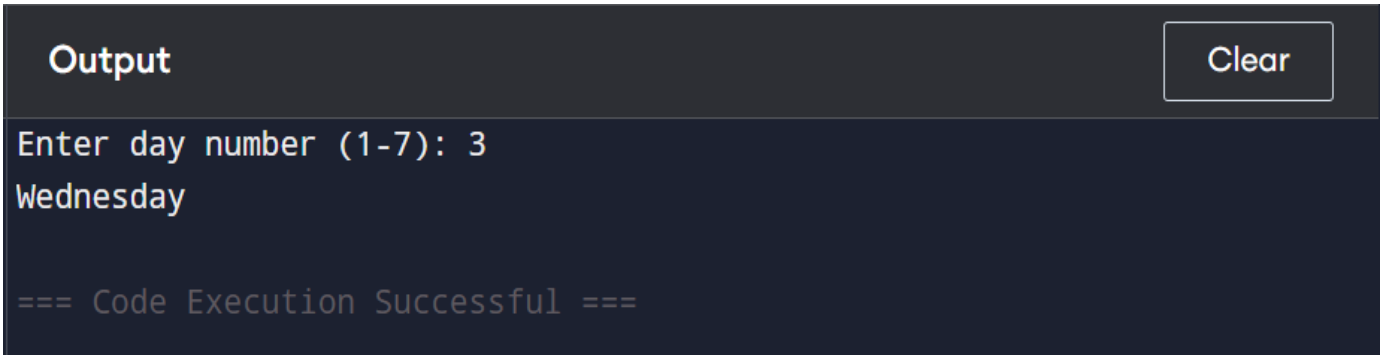
OUTPUT:

```
Output                                                    Clear

Enter first number: 18
Enter second number: 42
Enter third number: 47
Largest = 47

=== Code Execution Successful ===
```

CODE (D):

```
import java.util.Scanner;

class WeekDay {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      System.out.print("Enter day number (1-7): ");
      int day = sc.nextInt();

      switch (day) {
         case 1:
            System.out.println("Monday");
            break;
         case 2:
            System.out.println("Tuesday");
            break;
         case 3:
            System.out.println("Wednesday");
            break;
         case 4:
            System.out.println("Thursday");
            break;
         case 5:
            System.out.println("Friday");
            break;
         case 6:
            System.out.println("Saturday");
            break;
         case 7:
            System.out.println("Sunday");
            break;
         default:
            System.out.println("Invalid day number");
      }
   }
}
```

OUTPUT:


Output
Clear
Enter day number (1-7): 3
Wednesday
=== Code Execution Successful ===

PRACTICAL NO: 04

AIM:

Write a Java program to demonstrate looping constructs: a) Reverse of a number using while loop. b) Prime number using do while loop. c) nth term of fibonacci sequence using for loop.

THEORY:

Looping constructs in Java are used to execute a set of statements repeatedly until a specified condition is met. They reduce repetition and make programs easier to manage.

while Loop: It checks the condition before executing the loop body. It is used here to reverse a number.

do...while Loop: It executes the loop body at least once and checks the condition afterward. It is used here to check whether a number is prime.

for Loop: It is useful when the number of iterations is known. It is used here to calculate the required Fibonacci term.

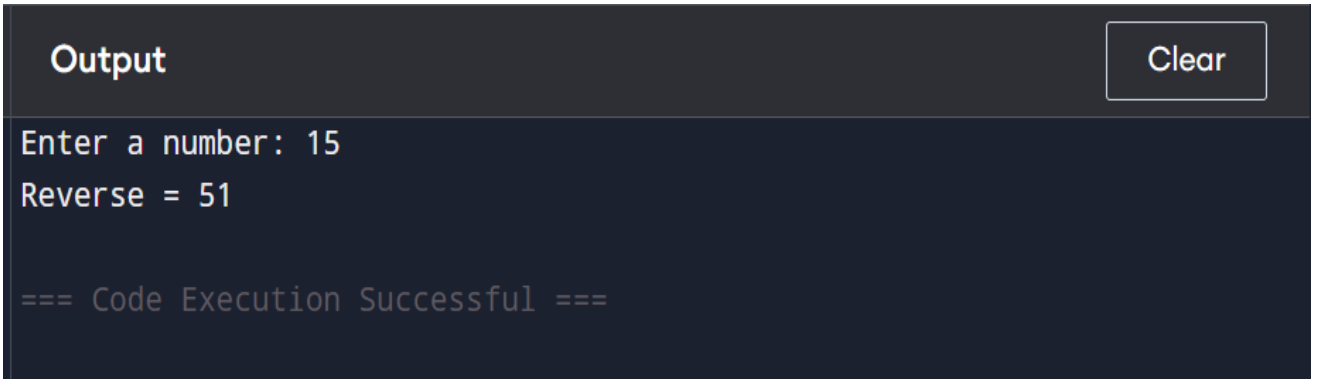
These loops demonstrate three different methods of controlling repeated execution in Java.

CODE (A):

```
import java.util.Scanner;

class ReverseNum {

   public static void
main(String[] args) {

    Scanner sc = new
Scanner(System.in);

      int n, rev = 0;
```

```
System.out.print("Enter a number: ");

      n = sc.nextInt();

      while (n != 0) {

         rev = rev * 10 + n % 10;

         n = n / 10;

}

    System.out.println("Reverse = " + rev);

  }
```

OUTPUT:

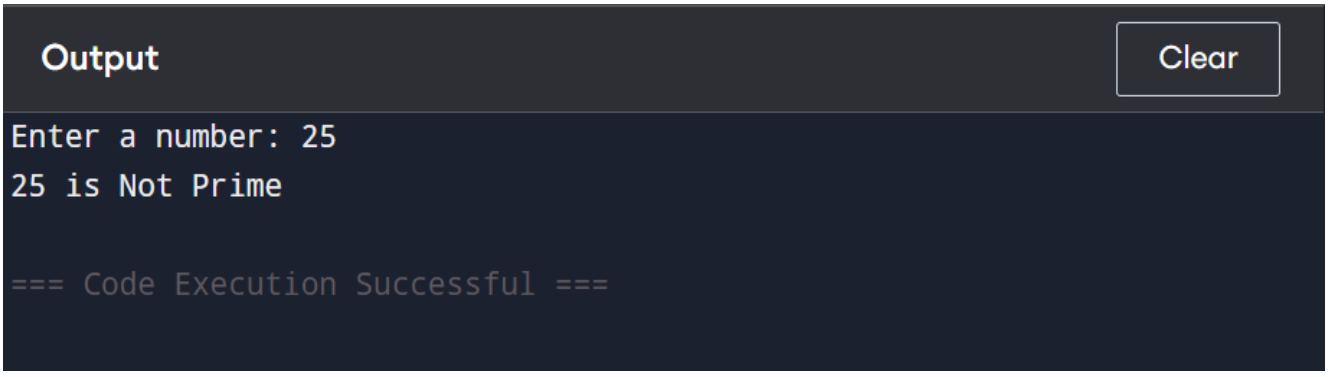


CODE (B):

```
import java.util.Scanner;

class PrimeNum {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);
      int n, i = 2, count = 0;
```

```
 System.out.print("Enter a number: ");
      n = sc.nextInt();

      if (n > 1) {
         do {
            if (n % i == 0)
               count++;
            i++;
         } while (i <= n / 2);
      }

      if (n > 1 && count == 0)
         System.out.println(n + " is Prime");
      else
         System.out.println(n + " is Not Prime");
   }
}
```

OUTPUT:

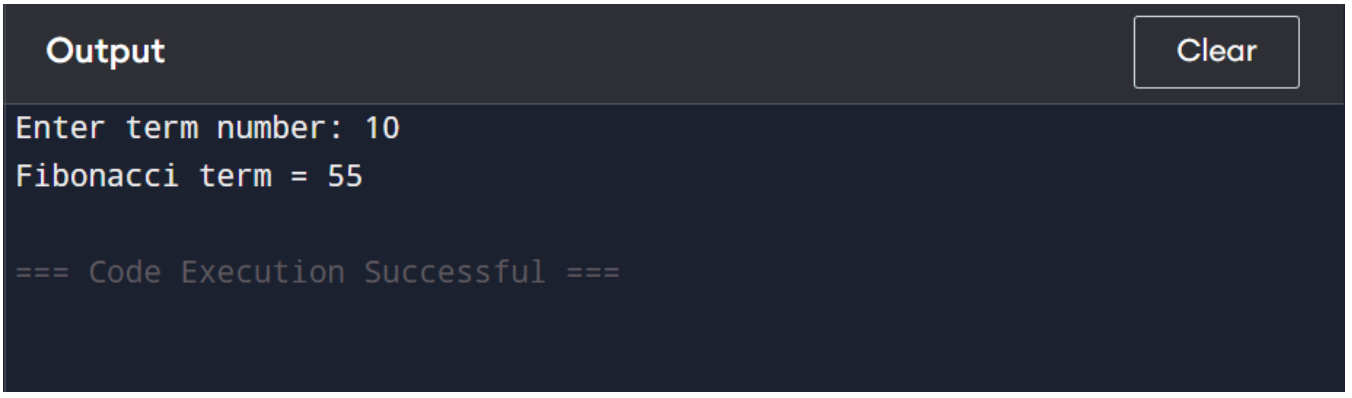


CODE (C):

```
import java.util.Scanner;

class Fibonacci {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);
      int n, a = 0, b = 1, c = 0;

      System.out.print("Enter term number: ");
      n = sc.nextInt();
```

```
    for (int i = 1; i < n; i++) {
            c = a + b;
            a = b;
            b = c;
        }

        System.out.println("Fibonacci term = " + (n == 1 ? a : b));
    }
}
```

OUTPUT:



PRACTICAL NO: 05

AIM:

Write a Java program to perform methods and example on recursion.

THEORY:

A **method** in Java is a separate block of statements created to perform a particular task. Methods make a program more organized, reusable, and easier to understand. A method can accept values as parameters and may return a result to the calling part of the program. In this practical, a method is used to calculate the square of a number.

Recursion is a process in which a method calls itself to solve a problem in smaller steps. A recursive method must have a **base condition** to stop the repeated calls. In this practical, recursion is demonstrated by calculating the power of a number, where the method repeatedly calls itself with a reduced power.

Thus, the practical demonstrates **methods and recursion separately** using two different programs.

CODE (A): Method Example :

```
import java.util.Scanner;

class MethodDemo {

    static int square(int n) {
        return n * n;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a number: ");
        int n = sc.nextInt();

        System.out.println("Square = " + square(n));
    }
}
```

OUTPUT:

```
Output                                                    Clear
Enter a number: 4
Square = 16

=== Code Execution Successful ===
```

CODE (B): Recursion Example

```
import java.util.Scanner;

class RecursionDemo {

    static int power(int n, int p) {
        if (p == 0)
            return 1;
        return n * power(n, p - 1);
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number: ");
        int n = sc.nextInt();

        System.out.print("Enter power: ");
        int p = sc.nextInt();

        System.out.println("Result = " + power(n, p));
    }
}
```

```
Output                                                    Clear
Enter number: 3
Enter power: 4
Result = 81

=== Code Execution Successful ===
```

PRACTICAL NO: 06

AIM:

Write a Java program to perform operations on one-dimensional and multi -dimensional arrays.

THEORY:

An **array** in Java is a collection of elements of the same data type stored under a single variable name. Arrays are useful for storing and processing multiple values without creating separate variables for each value.

1. One-Dimensional Array:

A **one-dimensional array** stores elements in a single row and each element is accessed using an index. The index starts from 0. It can be used to store values such as marks, numbers, or names.

2. Multi-Dimensional Array:

A **multi-dimensional array** stores data in more than one dimension. A two-dimensional array is commonly represented in the form of rows and columns, similar to a matrix. Elements are accessed using two indexes: row and column.

In this practical, separate programs are used to demonstrate operations on **one-dimensional and multi-dimensional arrays**.

CODE (A): One-Dimensional Array

```
import java.util.Scanner;

class OneDArray {

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      int[] a = new int[5];

      System.out.println("Enter 5 numbers:");

      for (int i = 0; i < 5; i++)

         a[i] = sc.nextInt();

      System.out.println("Array elements:");

      for (int i = 0; i < 5; i++)

         System.out.print(a[i] + " "); } }
```

OUTPUT

```
Output                                                    Clear
Enter 5 numbers:
10
30
20
50
40
Array elements:
10 30 20 50 40
=== Code Execution Successful ===
```

CODE (B): Multi-Dimensional Array

```
import java.util.Scanner;
class TwoDArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[][] a = new int[2][3];
        System.out.println("Enter 6 numbers:");
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 3; j++)
                a[i][j] = sc.nextInt();
        System.out.println("Matrix:");
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 3; j++)
                System.out.print(a[i][j] + " ");
            System.out.println();
        } } }
```

OUTPUT

Output Clear

```
Enter 6 numbers:
1
2
3
4
5
6
Matrix:
1 2 3
4 5 6

=== Code Execution Successful ===
```

PRACTICAL NO: 07

AIM:

Write a Java program to demonstrate 1D - Arrays & 2-D Arrays. a) Maximum value and Second Maximum value without duplicates. b) Sort the names in Ascending Order. c) Addition of two matrix. d) 3x3 Matrix Multiplication

THEORY:

Arrays in Java are used to store multiple values of the same data type under a single variable name. An array provides indexed access to its elements, with indexing beginning from 0. Java supports both **one-dimensional arrays** and **two-dimensional arrays**, which are useful for handling lists and tabular data respectively.

A **one-dimensional array** stores elements in a single sequence. It can be processed using loops to search, compare, and arrange its elements. In this practical, a 1D array is used to find the **maximum and second maximum values without duplicates** and to **sort names in ascending order**.

A **two-dimensional array** stores elements in rows and columns and is commonly used to represent matrices. Matrix operations can be performed by accessing elements using row and column indexes. In this practical, 2D arrays are used for **addition of two matrices** and **multiplication of two 3×3 matrices**.

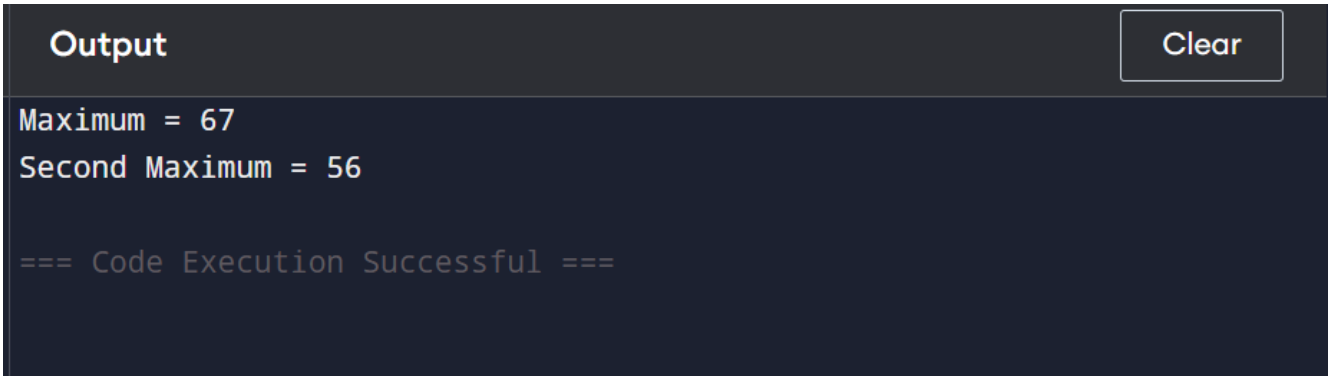
Thus, the practical demonstrates important array operations such as **searching, comparison, sorting, matrix addition, and matrix multiplication** using both 1D and 2D arrays. The manual specifically groups these four operations under the 1D and 2D array practical.

CODE (A):

```
class MaxValues {
   public static void main(String[] args) {

      int[] a = {12, 45, 23, 67, 34, 67, 56};
      int max = Integer.MIN_VALUE;
      int second = Integer.MIN_VALUE;

      for (int n : a) {
         if (n > max) {
            second = max;
            max = n;
```

```
    } else if (n > second && n != max) {
                second = n;
            }
        }

        System.out.println("Maximum = " + max);
        System.out.println("Second Maximum = " +
    second);
      }
    }
```

OUTPUT



CODE (B):

```
import java.util.Arrays;

class NameSort {
   public static void main(String[] args) {

      String[] names = {"Riya", "Aman", "Karan", "Neha", "Vikas"};

      Arrays.sort(names);

      System.out.println("Names in Ascending Order:");
      for (String name : names)
         System.out.println(name);
   }
}
```

OUTPUT

```
Output                                                    Clear

Names in Ascending Order:
Aman
Karan
Neha
Riya
Vikas

=== Code Execution Successful ===
```

CODE (C):

```
class MatrixAddition {
    public static void main(String[] args) {

        int[][] a = {{1, 2}, {3, 4}};
        int[][] b = {{5, 6}, {7, 8}};

        System.out.println("Matrix Addition:");

        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++)
                System.out.print((a[i][j] + b[i][j]) + " ");
            System.out.println();
        }
    }
}
```

```
Output                                                    Clear

Matrix Addition:
6 8
10 12

=== Code Execution Successful ===
```

CODE (D):

```
class MatrixMultiply {
    public static void main(String[] args) {

        int[][] a = {
            {1, 2, 1},
            {2, 1, 2},
            {1, 1, 1}
        };

        int[][] b = {
            {1, 1, 2},
            {2, 1, 1},
            {1, 2, 1}
        };

        int[][] c = new int[3][3];

        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 3; j++)
                for (int k = 0; k < 3; k++)
                    c[i][j] += a[i][k] * b[k][j];

        System.out.println("3x3 Matrix Multiplication:");

        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++)
                System.out.print(c[i][j] + " ");
            System.out.println();
        }
    }
```

Output Clear

```
3x3 Matrix Multiplication:
6 5 5
6 7 7
4 4 4

=== Code Execution Successful ===
```

PRACTICAL NO: 08

AIM:

Write a Java program to demonstrate string handling using String, StringBuffer, and StringBuilder.

THEORY:

In Java, strings are sequences of characters used to store and manipulate text. Java provides three commonly used classes for string handling: **String, StringBuffer, and StringBuilder**.

1. String

String is an immutable class. Once a String object is created, its value cannot be changed. Any modification creates a new String object. It provides methods such as length(), toUpperCase(), toLowerCase(), substring(), and concat().

2. StringBuffer

StringBuffer is a mutable class, which means its contents can be modified without creating a new object. It is **thread-safe** because its methods are synchronized. It provides methods such as append(), insert(), delete(), and reverse().

3. StringBuilder

StringBuilder is also a mutable class and provides operations similar to StringBuffer. However, its methods are **not synchronized**, so it is generally faster than StringBuffer when thread safety is not required.

Thus, String is preferred when the text does not need frequent modification, while StringBuffer and StringBuilder are useful when frequent modifications of strings are required.

CODE (A):

```
public class StringDemo {
    public static void main(String[] args) {

        String s = "Hello";
        System.out.println("String: " + s);

        StringBuffer sb = new StringBuffer("Hello");
        sb.append(" Java");
        System.out.println("StringBuffer: " + sb);
```

```
 StringBuilder sbd = new StringBuilder("Hello");
        sbd.append(" Java");
        System.out.println("StringBuilder: " + sbd);
    }
}
```

OUTPUT

```
Output

String: Hello
StringBuffer: Hello Java
StringBuilder: Hello Java

=== Code Execution Successful ===
```

PRACTICAL NO: 09

AIM:

Write a Java program to check the word is palindrome or not.

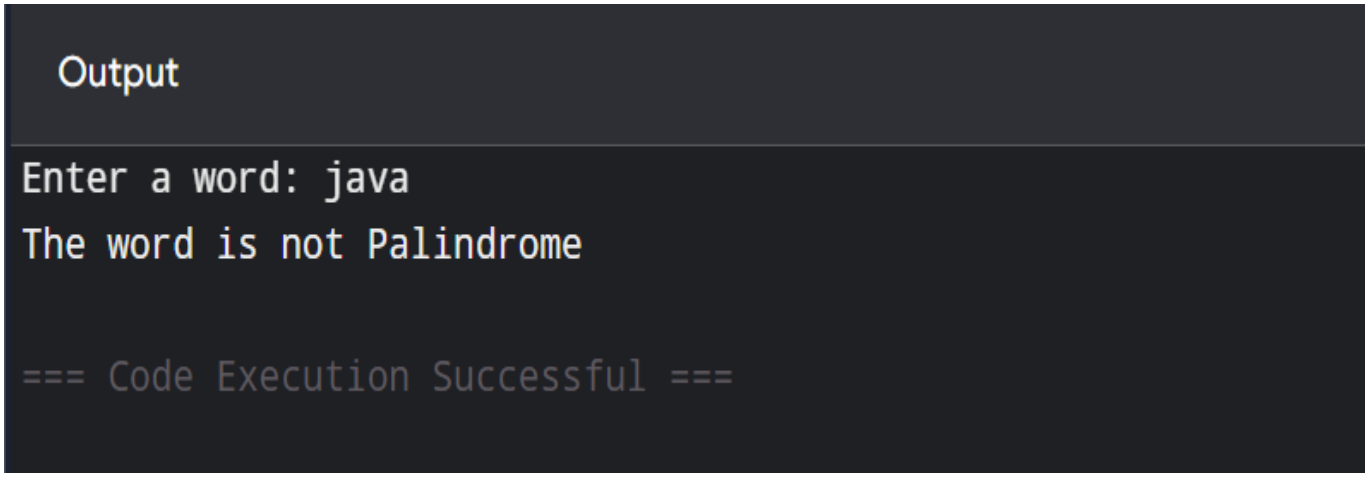
THEORY:

A palindrome is a word that reads the same from left to right and right to left. Examples of palindrome words are madam, level, radar.
In Java, a string can be reversed using the reverse() method of the StringBuilder class. The original word is then compared with the reversed word using equals(). If both are the same, the word is a palindrome.

CODE (A):

```
import java.util.Scanner;

public class Palindrome {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a word: ");
        String word = sc.nextLine();

        String reverse = new StringBuilder(word).reverse().toString();

        if (word.equals(reverse))
            System.out.println("The word is Palindrome");
        else
            System.out.println("The word is not Palindrome");
    }
}
```

OUTPUT


Output
Enter a word: java
The word is not Palindrome
=== Code Execution Successful ===

PRACTICAL NO: 10

AIM:

Write a Java program to create a class and object and demonstrate constructors.

THEORY:

A **class** is a blueprint that contains data members and methods. An **object** is an instance of a class used to access its members.
A **constructor** is a special method that has the same name as the class. It is automatically called when an object is created. Constructors are mainly used to initialize the data members of a class.
In Java, a constructor does not have a return type. A class can have a **default constructor** as well as a **parameterized constructor**.

CODE (A):

```
class student
{
   int marks;
   String name;
   public String toString(){
      return marks+" "+name;
   }
   student()
   {
   }
   student(int marks,String name)
   {
      this.name = name;
      this.marks = marks;
   }
}
class Main {
   public static void main(String[] args) {
      student s1 = new student(70,"Adi");
      student s2 = new student(80,"Kirtan");
      System.out.println(s1);
      System.out.println(s2);
   }
}
```

OUTPUT

```
Output

70 Adi
80 Kirtan

=== Code Execution Successful ===
```

PRACTICAL NO: 11

AIM:

Write a Java program to demonstrate encapsulation using access specifiers.

THEORY:

Encapsulation is the process of wrapping data and methods together inside a class. It helps protect the data from direct access.
Java provides access specifiers such as:
private – accessible only within the same class.
public – accessible from anywhere.
protected – accessible within the same package and through inheritance.
default – accessible within the same package.
In encapsulation, data members are generally declared as private and accessed using public methods such as getters and setters.

CODE (A):

```
class employe
{
   private int id;
   private String name;
   private int salary;

   public void setId(int id)
   {
      this.id = id;
   }

   public void setName(String name)
   {
      this.name = name;
   }
```

```
    public void setSalary(int salary)
    {
        this.salary = salary;
    }
    public int getID()
    {
        return id;
    }
    public String getName()
    {
        return name;
    }
    public int getSalary()
    {
        return salary;
    }
}
class Main {
    public static void main(String[] args) {
        employe e =new employe();
        e.setId(101);
        e.setName("Rohan");
        e.setSalary(25000);
        System.out.println(e.getID());
        System.out.println(e.getName());
        System.out.println(e.getSalary());
    }
}
```

Output

```
101
Rohan
25000

=== Code Execution Successful ===
```